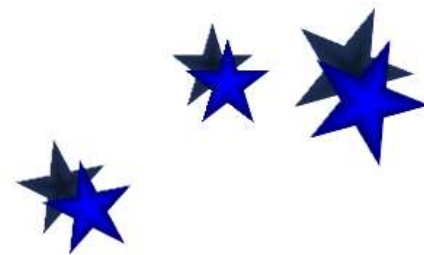


5.1 回溯算法的基本思想和适用条件

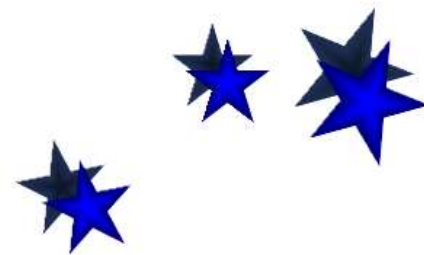
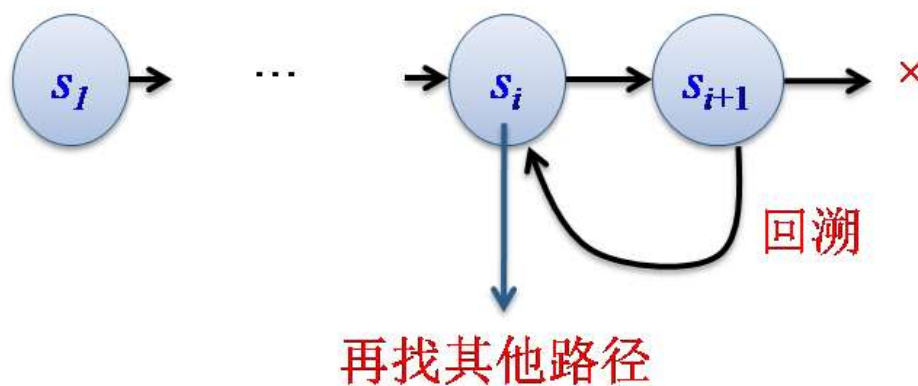
■ 什么是回溯法

- 在包含问题所有解的解空间树中，按照深度优先搜索的策略，从根结点（开始结点）出发搜索解空间树。
- 活结点、当前的扩展结点、死结点
- 从死结点处回溯至最近的一个活结点处，并使这个活结点成为当前的扩展结点。
- 回溯法以这种方式递归地在解空间中搜索，直至找到所要求的解或解空间中已无活结点为止。



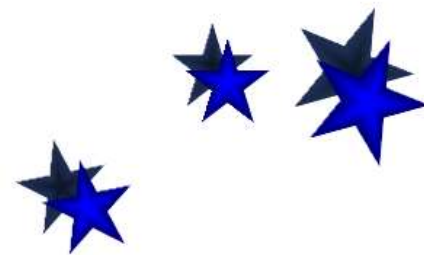
5.1 回溯算法的基本思想和适用条件

- 当从状态 s_i 搜索到状态 s_{i+1} 后，如果 s_{i+1} 变为死结点，则从状态 s_{i+1} 回退到 s_i ，再从 s_i 找其他可能的路径，所以回溯法体现出走不通就退回再走的思路。



5.1 回溯算法的基本思想和适用条件

- 在状态空间树中，每个叶子结点对应一个求解状态，每个非叶子结点对应一个部分解的状态。
- 解空间树是很规范的，数组 a 的元素个数为 n ，则解空间树的高度就为 $n+1$ 。第 i 层对应元素 a_i 的决策。
- 通常情况下，从根结点到叶子结点（不含搜索失败的结点）的路径构成了解空间的一个可行解。
- 求子集问题，属于求全部可行解问题，因此问题的解就是全部解空间；若问题修改为求子集最大和问题，就是一个求最优解问题，是解空间的一个子集。



5.1 回溯算法的基本思想和适用条件

- 回溯法搜索解空间时，通常采用两种策略避免无效搜索，提高回溯的搜索效率：

- 用**约束函数**在扩展结点处剪除不满足约束条件（例如：超过重量限制 W 、无法达到重量 W 等）的子树；
- 用限界函数剪去得不到问题解或最优解的子树（例如：某些子树**无法超越已得到的当前最优解**）——分支限界法。

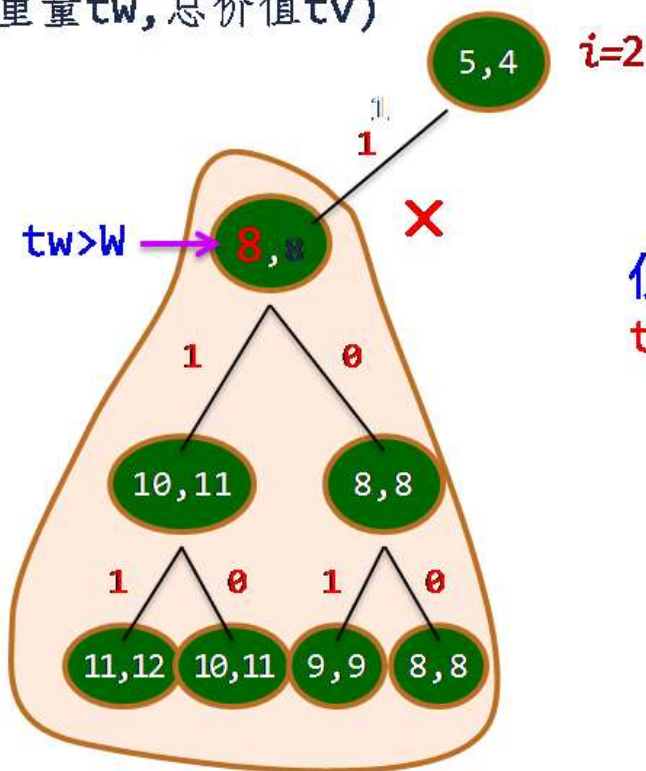
这两类函数统称为**剪枝函数**。



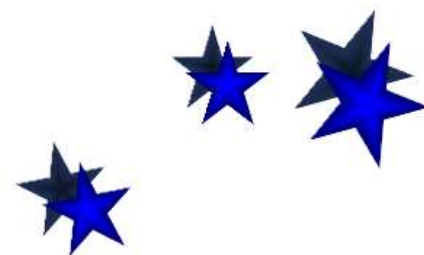
剪枝举例：对于第*i*层的有些结点， $tw+w[i]$ 已超过了 W （ $W=6$ ），显然再选择 $w[i]$ 是不合适的。如第2层的（5，4）结点 $\Rightarrow$ 进行扩展是不必要的。

(总重量 tw , 总价值 tv)

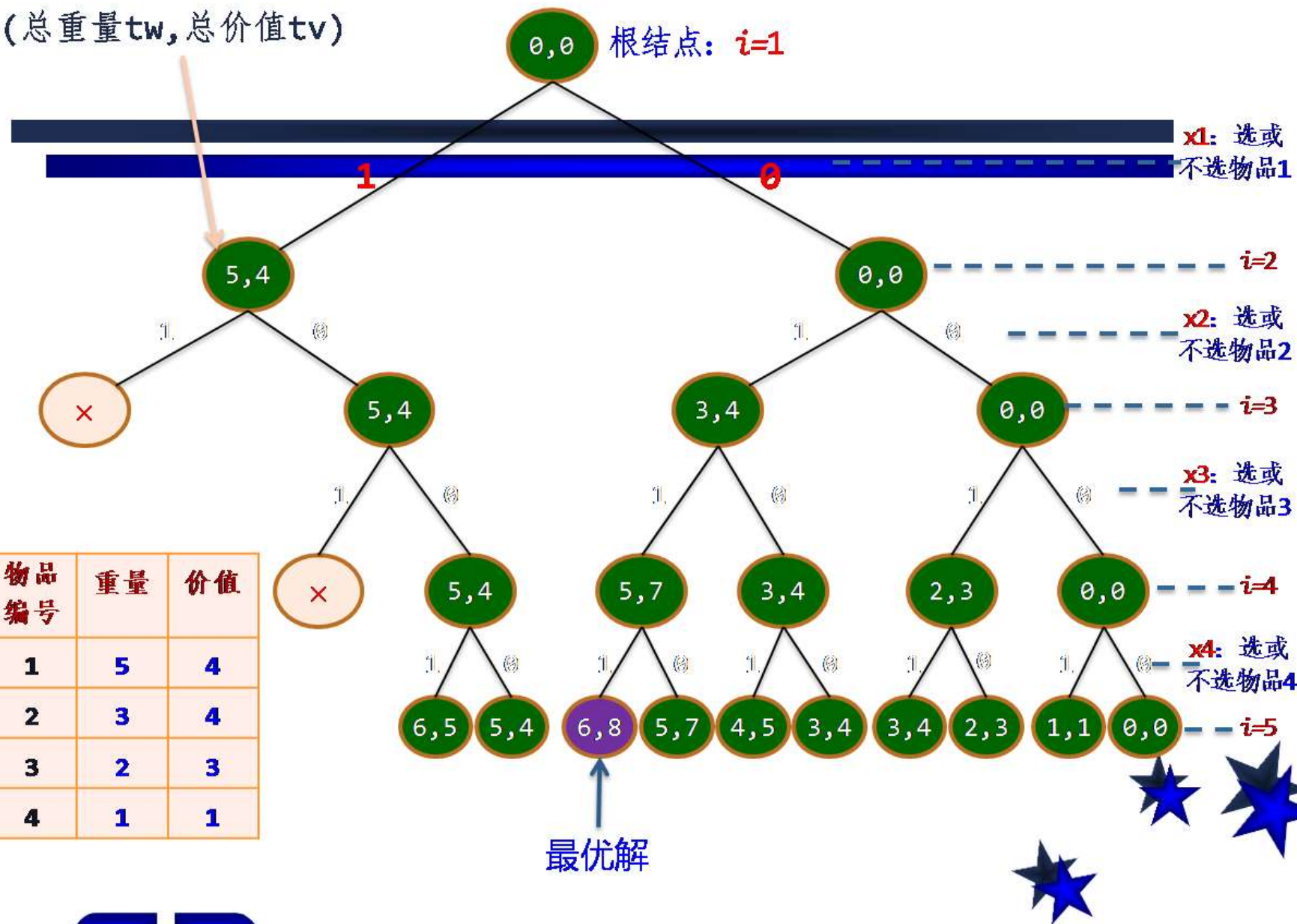
物品 编号	重量	价值
1	5	4
2	3	4
3	2	3
4	1	1



仅仅扩展满足
 $tw+w[i] \leq W$ 的左孩子结点



(总重量tw, 总价值tv)



5.1 回溯算法的基本思想和适用条件

归纳起来，用回溯法解题的一般步骤如下：

- ① 针对所给问题，确定问题的解空间树，问题的解空间树应至少包含问题的一个（最优）解。
- ② 确定结点的扩展搜索规则。
- ③ 以深度优先方式搜索解空间树，并在搜索过程中可以采用剪枝函数来避免无效搜索。



5.1 回溯算法的基本思想和适用条件

回溯算法的适用条件

在结点 $\langle x_1, x_2, \dots, x_k \rangle$ 处

$P(x_1, x_2, \dots, x_k)$ 为真

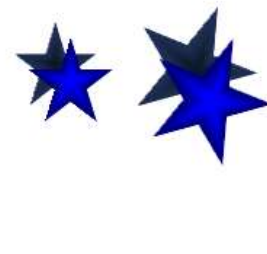
$\Leftrightarrow$ 向量 $\langle x_1, x_2, \dots, x_k \rangle$ 满足某个性质
(n 后中 k 个皇后放在彼此不攻击的位置)

多米诺性质

$$P(x_1, x_2, \dots, x_{k+1}) \rightarrow P(x_1, x_2, \dots, x_k) \quad 0 < k < n$$

$$\neg P(x_1, x_2, \dots, x_k) \rightarrow \neg P(x_1, x_2, \dots, x_{k+1}) \quad 0 < k < n$$

k 维向量不满足约束条件, 扩张向量到
 $k+1$ 维仍旧不满足, 可以回溯.



5.1 回溯算法的基本思想和适用条件

一个反例

例 求不等式的整数解

$$5x_1 + 4x_2 - x_3 \leq 10, \quad 1 \leq x_k \leq 3, \quad k=1,2,3$$

$P(x_1, \dots, x_k)$: 将 $x_1, x_2, \dots, x_k$ 代入原不等式的相应部分, 部分和小于等于10

不满足多米诺性质:

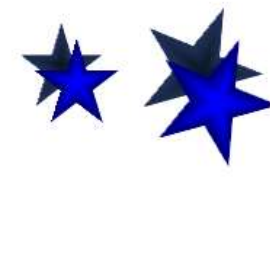
$$5x_1 + 4x_2 - x_3 \leq 10 \not\Rightarrow 5x_1 + 4x_2 \leq 10$$

变换使得问题满足多米诺性质:

令 $x_3 = 3 - x_3'$,

$$5x_1 + 4x_2 + x_3' \leq 13$$

$$1 \leq x_1, x_2 \leq 3, \quad 0 \leq x_3' \leq 2$$



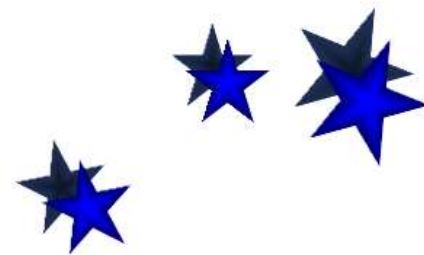
5.1 回溯算法的基本思想和适用条件

■ 回溯法与深度优先遍历的异同

- 两者的相同点：回溯法在实现上也是遵循深度优先的，即一步一步往前探索。

【举例】假设图G使用邻接表存储，设计一个算法输出图G中从顶点 u 到 v 的**所有**简单路径。

(1) 若仅使用深度优先遍历，则只能找到一条 u 到 v 的路径：因为每个访问过结点只访问一次($visit[u]=1$)



5.1 回溯算法的基本思想和适用条件

(2) 若使用回溯算法（深度遍历+回溯），才可以找到所有的 u 到 v 的路径，访问结点时 $visited[u]=1$ ，回溯到该结点时再将 $visited[u]=0$ 。

- 两者的不同点

(1) **访问次数的不同**：深度优先遍历对已经访问过的顶点不再访问，所有顶点**仅访问一次**。而回溯法中已经访问过的顶点可能**再次访问**（在回溯时将访问标识修改为1）。

(2) **剪枝的不同**：深度优先遍历不含剪枝，而很多回溯算法采用剪枝策略剪除不必要的分枝以提高算法效率。

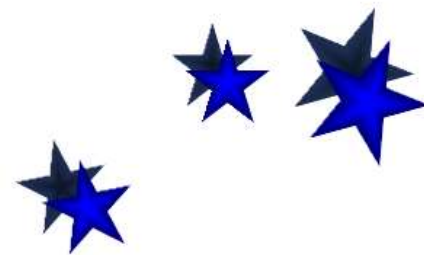


5.2 回溯算法的实现及实例

- 回溯算法的一般描述

- (1) 设解向量为 $\langle x_1, x_2, x_3, \dots, x_n \rangle$, 第 i 步确定 x_i 的值。
- (2) 初始时, x_i 的取值集合为 X_i 。
- (3) 当 $x_1, x_2, x_3, \dots, x_{k-1}$ 的值确定后, x_i 的取值集合为 S_i 。
- (4) $S_i \subseteq X_i$

- 举例: 4皇后问题
I



5.2 回溯算法的实现及实例

回溯算法递归实现

算法 **ReBack** (k)

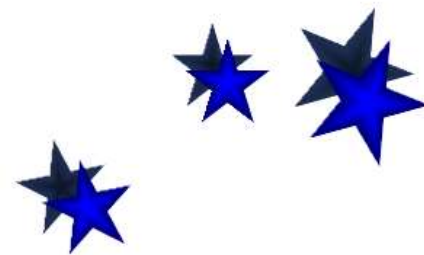
1. if $k > n$ then $\langle x_1, x_2, \dots, x_n \rangle$ 是解
2. else while $S_k \neq \emptyset$ do
3. $x_k \leftarrow S_k$ 中最小值
4. $S_k \leftarrow S_k - \{x_k\}$
5. 计算 S_{k+1}
6. **ReBack** ($k+1$)

算法 **ReBacktrack** (n)

输入: n

输出: 所有的解

1. for $k \leftarrow 1$ to n 计算 X_k 且 $S_k \leftarrow X_k$
2. **ReBack** (1)



5.2 回溯算法的实现及实例

迭代实现

迭代算法 Backtrack

输入: n

输出: 所有的解

1. 对于 $i=1, 2, \dots, n$ 确定 X_i
2. $k \leftarrow 1$
3. 计算 S_k
4. while $S_k \neq \emptyset$ do
5. $x_k \leftarrow S_k$ 中最小值; $S_k \leftarrow S_k - \{x_k\}$
6. if $k < n$ then
7. $k \leftarrow k+1$; 计算 S_k
8. else $\langle x_1, x_2, \dots, x_n \rangle$ 是解
9. if $k > 1$ then $k \leftarrow k-1$; goto 4

确定初始取值

满足约束
分支搜索

回溯

5.2 回溯算法的实现及实例

装载问题

问题：有 n 个集装箱，需要装上两艘载重分别为 c_1 和 c_2 的轮船。 w_i 为第 i 个集装箱的重量，且 $w_1+w_2+\dots+w_n \leq c_1+c_2$
问：是否存在一种合理的装载方案把这 n 个集装箱装上船？如果有，请给出一种方案。

实例：

$W = < \underline{90}, 80, \underline{40}, 30, 20, \underline{12}, 10 >$

$c_1=152, c_2=130$

解：1,3,6,7装第一艘船，其余第2艘船



5.2 回溯算法的实现及实例

求解思路

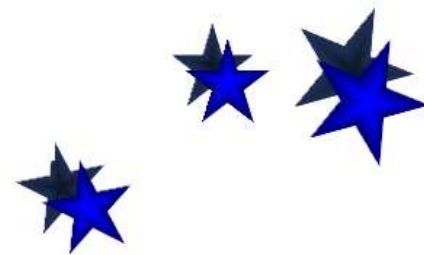
输入: $W = \langle w_1, w_2, \dots, w_n \rangle$ 为集装箱重量
 c_1 和 c_2 为船的最大载重量

算法思想: 令第一艘船的装入量为 W_1 ,

1. 用回溯算法求使得 $c_1 - W_1$ 达到最小的装载方案.
2. 若满足

$$w_1 + w_2 + \dots + w_n - W_1 \leq c_2$$

则回答 “Yes”, 否则回答 “No”



5.2 回溯算法的实现及实例

伪码

算法 Loading (W, c_1),

1. Sort(W);
2. $B \leftarrow c_1$; $best \leftarrow c_1$; $i \leftarrow 1$;
3. while $i \leq n$ do
4. if 装入 i 后重量不超过 c_1
5. then $B \leftarrow B - w_i$; $x[i] \leftarrow 1$; $i \leftarrow i + 1$;
6. else $x[i] \leftarrow 0$; $i \leftarrow i + 1$;
7. if $B < best$ then 记录解; $best \leftarrow B$;
8. Backtrack(i); 回溯
9. if $i = 1$ then return 最优解
10. else goto 3.

B为当前空隙
best 最小空隙



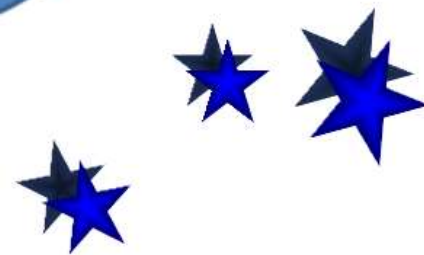
5.2 回溯算法的实现及实例

子过程 Backtrack

算法 Backtrack(i)

1. while $i > 1$ and $x[i] = 0$ do
2. $i \leftarrow i - 1$;
3. if $x[i] = 1$
4. then $x[i] \leftarrow 0$;
5. $B \leftarrow B + w_i$;
6. $i \leftarrow i + 1$.

沿右分支一
直回溯发现
左分支边，
或到根为止



5.2 回溯算法的实现及实例

实例

$$W = \langle 90, 80, 40, 30, 20, 12, 10 \rangle$$

$$c_1 = 152, c_2 = 130$$

解:

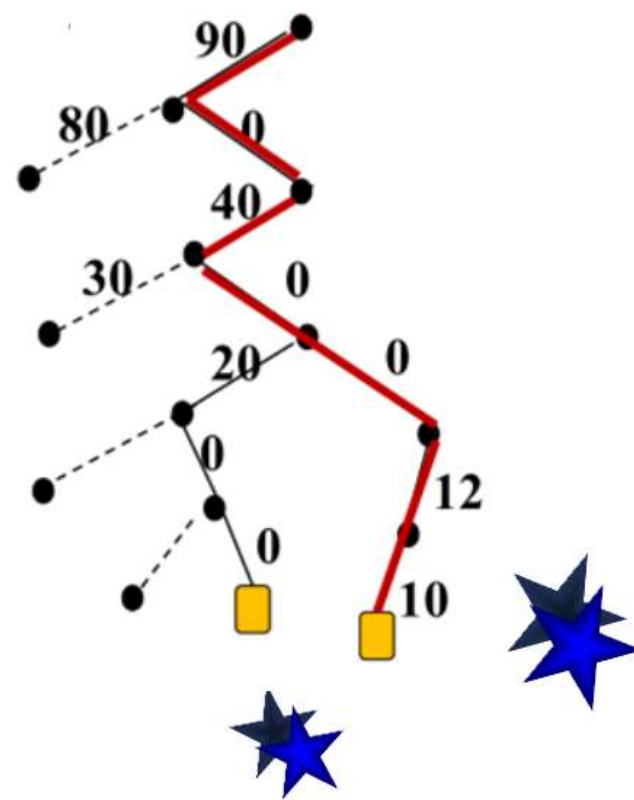
可以装, 方案如下:

1, 3, 6, 7 装第一艘船

2, 4, 5 装第二艘船

时间复杂性:

$$W(n) = O(2^n)$$



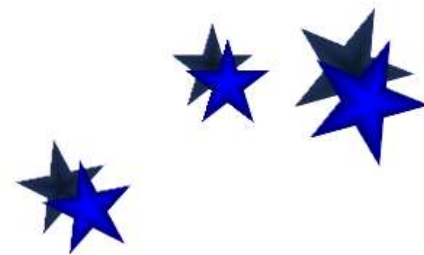
5.2 回溯算法的实现及实例

着色问题

输入:

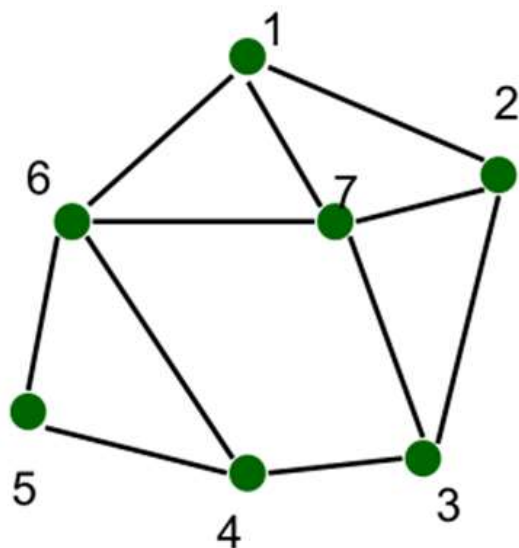
无向连通图 G 和 m 种颜色的集合
用这些颜色给图的顶点着色, 每个
顶点一种颜色. 要求是: G 的每条
边的两个顶点着不同颜色.

输出: 所有可能的着色方案.
如果不存在着色方案, 回答 “No”.

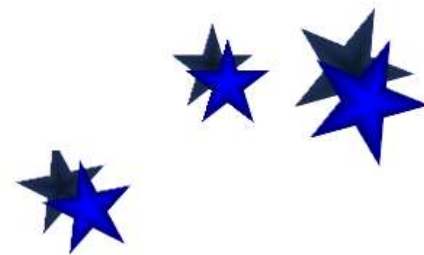
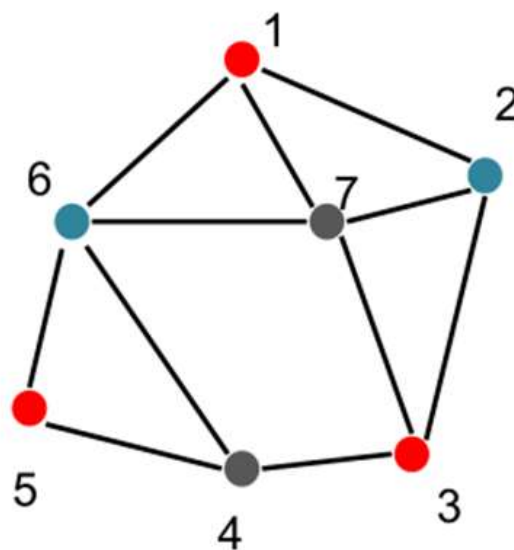


5.2 回溯算法的实现及实例

实例



$n=7, m=3$



5.2 回溯算法的实现及实例

解向量

设 $G=(V,E)$, $V=\{1,2,\dots,n\}$

颜色编号: $1, 2, \dots, m$

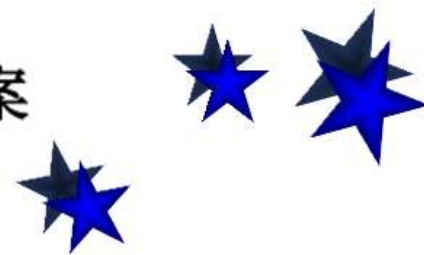
解向量: $\langle x_1, x_2, \dots, x_n \rangle$,

$$x_1, x_2, \dots, x_n \in \{1, 2, \dots, m\}$$

结点的部分向量 $\langle x_1, x_2, \dots, x_k \rangle$

$$x_1, x_2, \dots, x_k, 1 \leq k \leq n,$$

表示只给顶点 $1, 2, \dots, k$ 着色的部分方案



5.2 回溯算法的实现及实例

算法设计

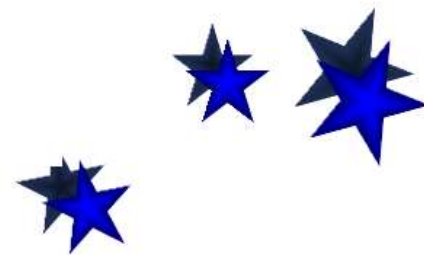
搜索树： m 叉树

约束条件：在结点 $\langle x_1, x_2, \dots, x_k \rangle$ 处，
顶点 $k+1$ 的邻接表中结点已用过的颜色
不能再用

如果邻接表中结点已用过 m 种颜色，
则结点 $k+1$ 没法着色，从该结点回溯
到其父结点。满足多米诺性质

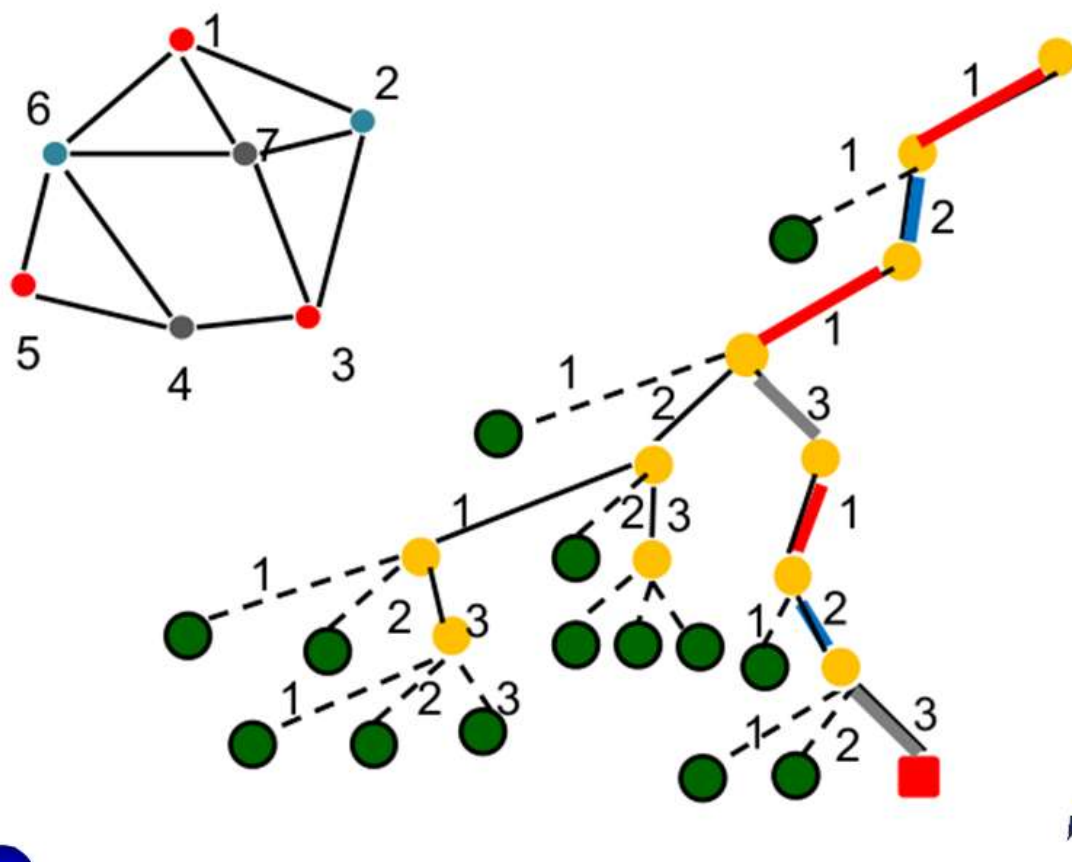
搜索策略：深度优先

时间复杂度： $O(n m^n)$

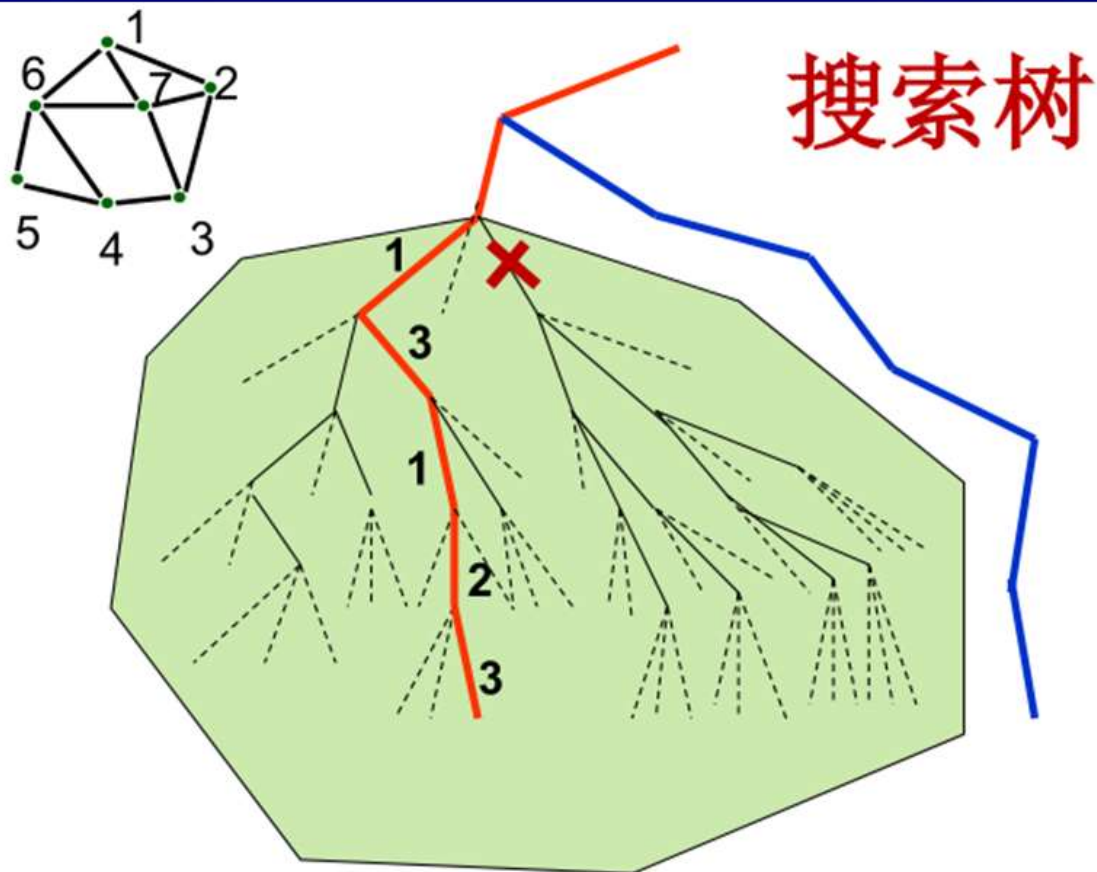


5.2 回溯算法的实现及实例

运行实例



5.2 回溯算法的实现及实例



第一个解向量: $\langle 1, 2, 1, 3, 1, 2, 3 \rangle$

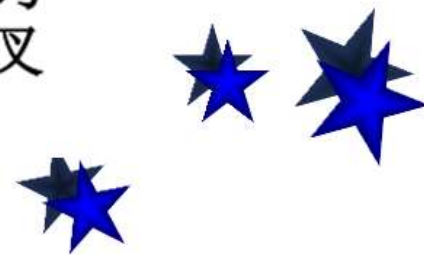
5.2 回溯算法的实现及实例

时间复杂度与 改进途径

时间复杂度: $O(nm^n)$

根据对称性, 只需搜索 $1/3$ 的解空间. 当 1 和 2 确定, 即 $\langle 1, 2 \rangle$ 以后, 只有 1 个解, 在 $\langle 1, 3 \rangle$ 为根的子树中也只有 1 个解. 由于 3 个子树的对称性, 总共 6 个解.

在取定 $\langle 1, 2 \rangle$ 后, 不可扩张成 $\langle 1, 2, 3 \rangle$, 因为 7 和 1, 2, 3 都相邻. 7 没法着色. 可以从打叉的结点回溯, 而不必搜索其子树.



5.2 回溯算法的实现及实例

着色问题的应用

会场分配问题：

有 n 项活动需要安排, 对于活动 i, j , 如果 i, j 时间冲突, 就说 i 与 j 不相容. 如何分配这些活动, 使得每个会场的活动相容且占用会场数最少?

建模：

活动作为图的顶点, 如果 i, j 不相容, 则在 i 与 j 之间加一条边, 会场标号作为颜色标号. 求图的一种着色方案, 使得使用的颜色数最少.

